

2장 공간 데이터 준비, 좌표계에서 거리 정의까지

- 이 장에 나오는 좌표값, 거리, 권역 면적, 건물 수는 모두 `lecture_practice/chapter2/` 의 코드를 실제로 실행해 얻은 값
- 예시를 위해 지어낸 숫자는 없음

1. 이 장의 핵심 질문

- 이 장은 세 가지 질문에 답함
- **질문 1** — "학교에서 반경 3km", "매장에서 반경 3km"라고 말할 때, 그 3km는 어디서 어떻게 재는 3km인가?
- **질문 2** — 직선으로 3km와 도로를 따라 3km는 얼마나 다른가? 그 차이는 무엇으로 재는가?
- **질문 3** — 좌표계를 잘못 쓴 계산은 왜 오류 메시지 없이 통과하는가? 무엇을 확인해야 잡을 수 있는가?
- 이 장은 분석을 시작하기 전에 **자료를 한 벌로 맞추고 거리를 정의하는 작업**을 다룸
- 여기서 생긴 오류는 프로그램이 잡아 주지 않음. 계산은 끝까지 돌고, 틀린 숫자가 최종 결과에만 남음
- 핵심은 다음 한 문장
- 좌표계를 정하지 않으면 거리도 면적도 계산할 수 없고, 계산해도 그 값이 무엇을 뜻하는지 말할 수 없음

2. 같은 3km가 네 개의 권역이 되는 사례

- 서울 옥수동의 한 매장이 "중심에서 반경 3km"를 배달 권역으로 정했다고 하면
- 실제 도로망과 건물 자료로 이 권역을 계산해 보면 결과가 하나가 아님
- 올바른 미터 좌표계에서 직선 3km 원을 그리면 건물 10,343 채가 들어옴
- 그런데 그중 7,925 채(76.6%)는 도로를 따라 재면 3km를 넘음
- 옥수동은 남쪽이 한강, 북쪽이 남산 자락. 직선으로 가까워도 다리까지 돌아가야 하는 건물이 많음
- 반대로 직선 3km 밖인데 큰길이 곧게 뚫려 10분 안에 도달하는 건물도 11채 있음
- 같은 "3km"인데 어느 좌표계에서 재는가, 직선인가 도로인가에 따라 권역이 전혀 달라짐
- 원을 그리는 계산 자체는 어느 방식이든 오류 없이 끝남. 무엇이 맞는지는 사람이 정해야 함

- 이 장 전체가 이 사례 하나를 이해하는 데 필요한 도구 — 좌표계, 좌표 변환, 공간 연산, 거리 정의 — 를 순서대로 갖추는 과정임

3. 실습 안내

- 실습 코드
 - `lecture_practice/chapter2/code/2-1-coordinate-transform.py` — 좌표 변환과 검증
 - `lecture_practice/chapter2/code/2-2-spatial-operations.py` — 버퍼·오버레이·공간 조인
 - `lecture_practice/chapter2/code/2-3-delivery-service-area.py` — 배달 권역 비교
- 결과 로그: `lecture_practice/chapter2/results/*.log`
- 이 장의 실습은 GPU가 필요 없고, 본문의 실행 수치는 전부 이 폴더의 결과 로그에서 나옴
- 도로망과 건물 자료는 OpenStreetMap에서 미리 내려받아 저장한 **스냅샷**을 읽음
- 매번 인터넷에서 받지 않으므로, 실행 시점이 달라도 같은 값이 나옴

4. 좌표계: 숫자의 단위를 정하는 규칙

4.1 지리 좌표계와 투영 좌표계

- **좌표참조체계**(Coordinate Reference System, CRS)는 지구 위 위치를 수치로 적는 규칙
- 좌표계마다 고유 번호(EPSS 코드)가 있고, 번호 하나에 기준·투영법·단위·적용 범위가 함께 정의됨
- 좌표계는 두 종류로 나뉨
- **지리 좌표계** — 경도·위도로 적음. 단위가 도(degree). 대표가 EPSG:4326(WGS84)
- **투영 좌표계** — 둥근 지구를 평면에 펴서 적음. 단위가 미터. 계산은 여기서 함
- 도(degree)를 거리 계산에 그대로 쓰면 안 되는 이유가 셋
 1. **경도 1도의 실제 거리가 위도에 따라 달라짐** — 적도에서 약 111km, 서울 위도에서 약 88km
 2. **위도 1도와 경도 1도의 길이가 다름** — 정사각형이라 믿고 계산하면 거리·면적이 틀어짐
 3. **도를 더하고 빼는 계산에 물리적 의미가 없음** — "0.02695도 버퍼"가 몇 미터인지 말할 수 없음
- 같은 서울 시청도 좌표계에 따라 전혀 다른 숫자로 적힘
- "어느 숫자가 맞는가"가 아니라 **"어느 좌표계에서 적은 숫자인가"**가 올바른 질문

4.2 어느 좌표계를 고를까

- 정답이 하나가 아니고 **분석 목적이 정함**

목적	고를 것	이유
한국 전역 분석	EPSG:5179 (Korea 2000 / Unified)	한국 전역을 하나의 평면으로 덮음, 통계청 표준
좁은 지역 정밀 분석	UTM 존 (EPSG:32652 등)	존 안에서 왜곡이 작음
면적 합산·밀도 계산	등면적 투영 (Mollweide 등)	면적 비율이 보존됨
웹 지도 배경	EPSG:3857 (웹 메르카토르)	타일 지도 서비스 표준. 계산에는 쓰지 않음
자료 교환·저장	EPSG:4326 (WGS84)	GPS·자료 교환의 국제 표준. 계산에는 쓰지 않음

- 마지막 두 행이 자주 사고가 나는 곳
- 자료를 주고받을 때는 4326이 표준이지만 **계산은 미터 좌표계로 바꾼 뒤에 함**
- 웹 메르카토르는 단위가 미터라서 더 위험함. 미터처럼 보이지만 위도가 올라갈수록 축척이 부풀려져, 서울에서 `buffer(3000)` 을 하면 3km보다 훨씬 작은 권역이 나옴 (8절에서 실측으로 확인)

4.3 UTM과 False Easting: 실습 값으로 확인

- `2-1-coordinate-transform.log` 의 실제 실행 결과
- 서울 (`127.0°E, 37.5°N`) → UTM 52N (`323,210.47m E, 4,152,220.15m N`)
- 부산 (`129.0°E, 35.1°N`) → UTM 52N (`500,000.00m E, 3,884,132.78m N`)
- 부산의 동쪽 좌표가 정확히 `500,000.00` 인 것이 눈에 띄. 우연이 아님
- **UTM**은 지구를 경도 6도 간격의 존(zone)으로 나누고, 존마다 평면 좌표를 매김. 한국 대부분은 52N 존 (중심 경도 129°E)
- 각 존의 중심 경도에 **500,000m를 부여**해 존 서쪽 절반의 좌표가 음수가 되지 않게 함. 이 보정값이 **False Easting**
- 부산이 52존의 중심 경도(129°E)에 거의 정확히 놓여 있어 500,000.00이 나옴
- 서울은 중심 경도보다 서쪽이라 500,000보다 작은 323,210.47이 나옴

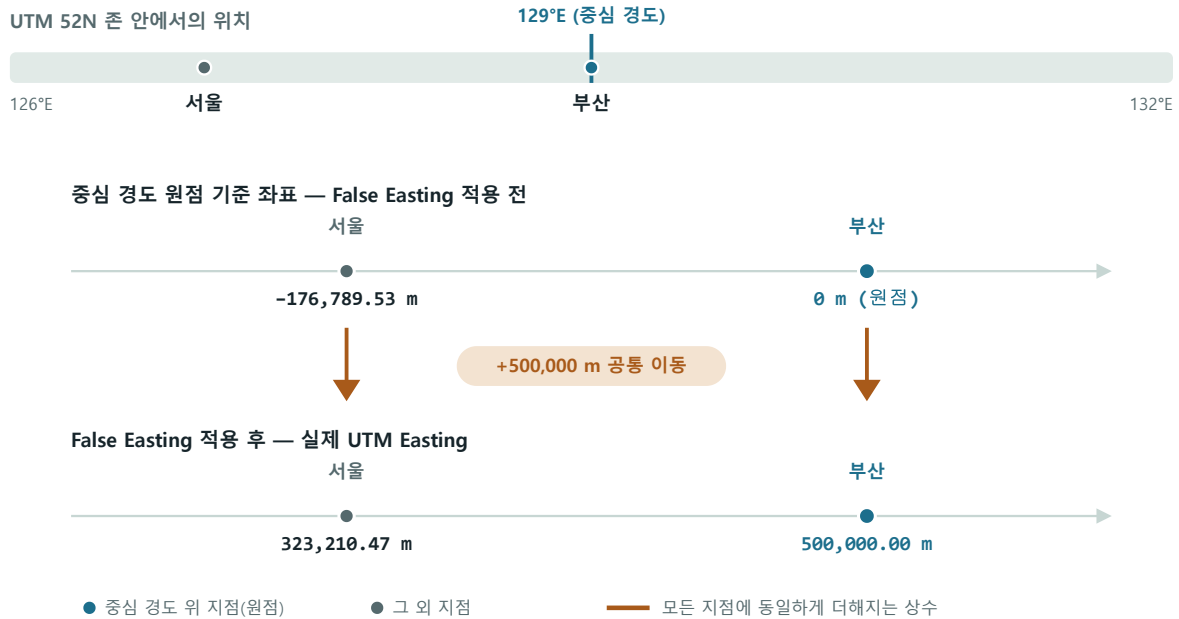
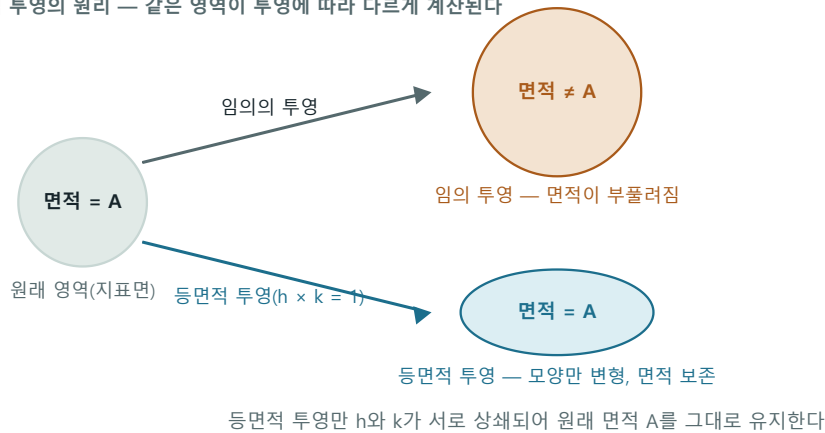


그림 2.1 중심 경도에 500,000m를 더하는 과정. 부산은 원점 위라 0m가 500,000.00m로, 서울은 원점 서쪽이라 음수 좌표가 323,210.47m로 바뀜.

4.4 면적은 등면적 투영에서 계산한다

- 면적 계산은 좌표계 함정이 가장 뚜렷하게 드러나는 곳
- 실행 결과에서 한국 경계의 면적을 경위도 좌표(EPSG:4326)로 계산하면 9.9548 이 나옴
- 이 값의 단위는 도²(도의 제곱)이며, 제곱킬로미터가 아님. 어떤 실제 면적과도 비교할 수 없는 숫자
- 같은 경계를 등면적 투영(Mollweide)으로 옮겨 계산하면 99,206km² 가 나옴
- 두 계산 모두 오류 없이 끝남. 하나는 쓸 수 없는 값이고 하나는 쓸 수 있는 값인데, 프로그램은 구분해 주지 않음
- 등면적 투영은 면적 비율이 보존되도록 만든 좌표계
- 투영은 무엇을 살리면 무엇을 포기함 — 각도를 살리면 면적이 틀어지고(UTM·웹 메르카토르), 면적을 살리면 모양이 틀어짐(Mollweide)
- 그래서 면적을 잴 때와 거리를 잴 때 좌표계를 다르게 고르는 것이 정상임

등면적 투영의 원리 — 같은 영역이 투영에 따라 다르게 계산된다



1단계 · 경위도 좌표에서 그대로 계산

9.9548 도²

면적 단위 아님 · 비교 불가



Mollweide 등면적 투영으로 재계산

2단계 · 등면적 투영(Mollweide) 좌표에서 계산

99,206 km²

물리적 면적



통계 참고값과 대조

3단계 · 통계청 공표 참고값과 비교

100,210 km²

차이 약 1,004 km²(≈1%)

원인: 경계 자료의 단순화
(등면적 투영 자체의 오차 아님)

그림 2.2 면적을 보존하는 투영의 원리와 한국 경계 자료의 실제 계산 결과. 9.9548도²는 단위가 도²라 비교 불가이고, Mollweide로 계산한 99,206km²가 쓸 수 있는 값임.

5. 좌표 변환과 세 가지 검증

- **재투영(reprojection)**은 한 좌표계의 좌표를 다른 좌표계로 다시 계산하는 연산
- GeoPandas에서는 `to_crs()` 한 줄

```
gdf_utm = gdf.to_crs("EPSG:32652") # 경위도 → UTM 52N (미터)
```

- 변환 자체는 정보를 잃지 않음. 실행 결과에서 서울 좌표를 UTM으로 갔다가 되돌리면 원본과의 차이가 0.000000000000014도 — 사실상 0
- 문제는 변환 계산이 아니라 **변환을 안 했거나, 잘못된 좌표계로 했을 때** 생김
- 옮긴 뒤에는 세 가지로 검증함
 1. **역변환 오차** — 다시 원래 좌표계로 되돌려 원본과 비교. 위의 0.000000000000014도가 그 값
 2. **축 순서 확인** — 어떤 좌표계는 (경도, 위도) 순이고 어떤 것은 (위도, 경도) 순. 뒤바뀌면 한국이 바다 한가운데로 감

3. 지도 육안 확인 — 옮긴 레이어를 배경지도에 그려 제자리에 있는지 봄

주의. 축 순서가 뒤바뀐 변환도 오류 없이 끝나고, 좌표값도 그럴듯한 범위에 남음. 공공데이터의 좌표 컬럼명은 `X/Y`, `lon/lat`, 경도/위도가 섞여 있고 어느 것이 어느 축인지 파일에 적혀 있지 않은 경우가 많음. 변환 후에는 반드시 점 몇 개를 지도에 찍어 위치를 눈으로 확인함

- 변환이 맞았는지 가늠하는 또 다른 방법은 **아는 거리와 대조하는 것**
- 같은 두 도시의 거리를 UTM 평면에서 잴 값과 지구 곡면을 따라 잴 값(Haversine)으로 비교하면

구간	UTM 평면거리	Haversine(구면거리)	차이
서울-부산	321.13km	321.45km	0.10%
서울-제주	446.16km	447.07km	0.20%
서울-인천	27.11km	27.05km	0.22%

- 차이가 0.1~0.2%로 작음. **한반도 규모에서는 UTM 평면거리를 써도 실무상 문제가 없다는 뜻**
- 두 값이 크게 벌어졌다면 좌표계 지정이나 변환이 잘못된 것

6. 벡터 공간 연산 세 가지

6.1 버퍼: 거리의 단위는 좌표계가 정한다

- **버퍼(buffer)**는 점·선·면에서 일정 거리 안의 영역을 만드는 연산. "학교 반경 500m", "매장 반경 3km"가 전부 버퍼
- 이 장에서 가장 자주 사고가 나는 한 줄 — **버퍼 거리의 단위는 좌표계가 정함**
- 경위도 좌표계에서 `buffer(3000)` 을 하면 3,000미터가 아니라 **3,000도**짜리 버퍼가 만들어짐. 지구 둘레를 여러 바퀴 도는 크기
- 이때 GeoPandas는 오류가 아니라 경고만 냄. 실행 로그에 실제로 찍힌 문장:

```
Geometry is in a geographic CRS. Results from 'buffer' are likely incorrect.
Use 'GeoSeries.to_crs()' to re-project geometries to a projected CRS before this operation.
```

- 경고는 화면을 지나가고 코드는 끝까지 뚫. 결과 파일도 만들어짐
- 올바른 순서는 두 단계: ① `to_crs()` 로 미터 좌표계로 옮김 → ② `buffer(3000)` 을 함
- 1장에서 "결과가 나왔다고 맞는 것이 아니다"를 확인했음. 이 경고가 그 대표 사례임

6.2 오버레이: 두 레이어를 겹쳐 자른다

- **오버레이(overlay)**는 두 폴리곤 레이어를 겹쳐 집합 연산을 계산함
- **intersection** (교집합), **union** (합집합), **difference** (차집합)
- 산출물은 원본 두 레이어의 속성을 모두 물려받은 새 폴리곤 집합 — 조각 하나하나가 "A의 어느 폴리곤이면서 B의 어느 폴리곤인가"를 담고 있음
- 실행 결과에서 한국 경계와 다른 폴리곤의 교집합 면적이 **99,206km²** 로 나온 것이 이 연산임. 침수구역과 건물을 겹치면 "침수구역 안의 건물"만 남길 수 있음

6.3 공간 조인: 위치를 키로 결합한다

- **공간 조인(spatial join)**은 속성 값이 아니라 **공간 관계**를 키로 삼아 두 레이어를 결합함
- **within** (안에 있음), **intersects** (겹침), **contains** (포함함)
- 건물 점 수만 개에 행정동 코드를 붙이는 작업이 전형적 사례. 두 자료에 공통 키가 없으므로 위치로 판정함

주의. 조인 뒤에는 **행 수를 반드시 확인함**. 점이 두 폴리곤 경계에 걸치면 행이 늘어나고, 어느 폴리곤에도 안 들어가면 행이 줄어듦. 조인 전후 행 수가 다르면 그 이유를 설명할 수 있어야 함. 이것도 오류 메시지 없이 지나가는 함정임

7. 직선거리와 도로망 거리

7.1 세 가지 거리

- "중심에서 얼마 이내"를 계산으로 옮기려면 거리를 무엇으로 잴지 먼저 정해야 함

거리	정의	쓰는 곳
직선 거리	두 점을 잇는 최단 직선	빠른 1차 선별
네트워크 거리	도로를 따라간 최단 경로 길이	실제 이동
통행시간	네트워크 거리 ÷ 속도	배달 약속, 대응 시간

- 네트워크 거리를 계산하는 절차는 세 단계
 1. 도로 교차점을 점으로, 도로 구간을 선으로 삼아 **도로망 그래프**를 만들
 2. 매장과 건물을 각각 가장 가까운 교차점에 붙임
 3. 매장 교차점에서 모든 교차점까지의 최단 경로 길이를 한 번에 계산함

- 도로를 따라간 거리는 직선보다 짧을 수 없음
- 따라서 "주행 3km 안" 건물은 언제나 "직선 3km 안" 건물의 일부임. 도로망으로 재면 권역은 커질 수 없고 같거나 작아지기만 함
- 직선 반경은 구조적으로 과대 포함이고, 남은 질문은 "얼마나 과대인가"임

7.2 우회비: 과대 포함을 재는 숫자

- 두 거리의 비율이 **우회비**(circuitry) = 네트워크 거리 ÷ 직선 거리
- 우회비 1.3이면 직선 1km 거리를 도로에서는 1.3km 주행한다는 뜻. 1보다 작을 수 없음
- 완전한 바둑판 격자 도로망의 이론 평균은 $4/\pi \approx 1.273$
- 2-3-delivery-service-area.log 의 실측 — 옥수 지점, 교차점 14,035 개·구간 37,193 개·건물 37,717 채 규모
- **우회비 중앙값 1.493, 90분위 1.703**
- 이론값 1.273보다 크고, 상위 10% 건물은 직선의 1.7배를 넘게 주행함
- 실제 도로망이 완전한 격자가 아니기 때문. 강, 철로, 막다른 길, 일방통행이 경로를 늘림
- 같은 계산을 서울의 세 지점에서 반복하면 우회비가 지점마다 다름

지점	우회비 중앙값	90분위	지형 조건
옥수	1.493	1.703	남쪽 한강 + 북쪽 남산 자락
신림	1.270	1.526	남쪽 관악산
화곡	1.327	1.517	큰 장애물 없는 평지 격자

- 강과 산이 도로를 끊는 옥수의 우회비가 가장 크고, 평지 격자인 화곡은 이론값에 가까움
- **우회비는 지역의 상수가 아니라 지점마다 재야 하는 값임.** 같은 반경 규칙을 전 지점에 적용하면 손해가 특정 지점에 몰림
- 옥수는 중앙값과 90분위의 격차(0.210)도 가장 큼. 방향에 따라 도로 사정이 크게 다르다는 신호

8. 권역 네 방식의 비교와 반경 결정

8.1 네 방식을 실측으로 비교

- 옥수 지점 하나에 대해 "반경 3km" 권역을 네 방식으로 그림
- **B(올바른 미터 좌표계 EPSG:5179에서 3km 버퍼)**를 기준으로 삼음

권역	면적	B 대비	건물 수	B 대비	방식
A1	22.33 km ²	-20.9%	6,473채	-37.4%	경위도에 0.02695° 버퍼
A2	17.70 km ²	-37.3%	4,062채	-60.7%	웹 메르카토르에 buffer(3000)
B	28.23 km²	기준	10,343채	기준	EPSG:5179에 buffer(3000)
C	14.22 km ²	-49.6%	2,418채	-76.6%	도로망 3,000m 도달권
D	17.97 km ²	-36.3%	3,978채	-61.5%	주행 10분 등시권(20km/h 가정, 3,333m)

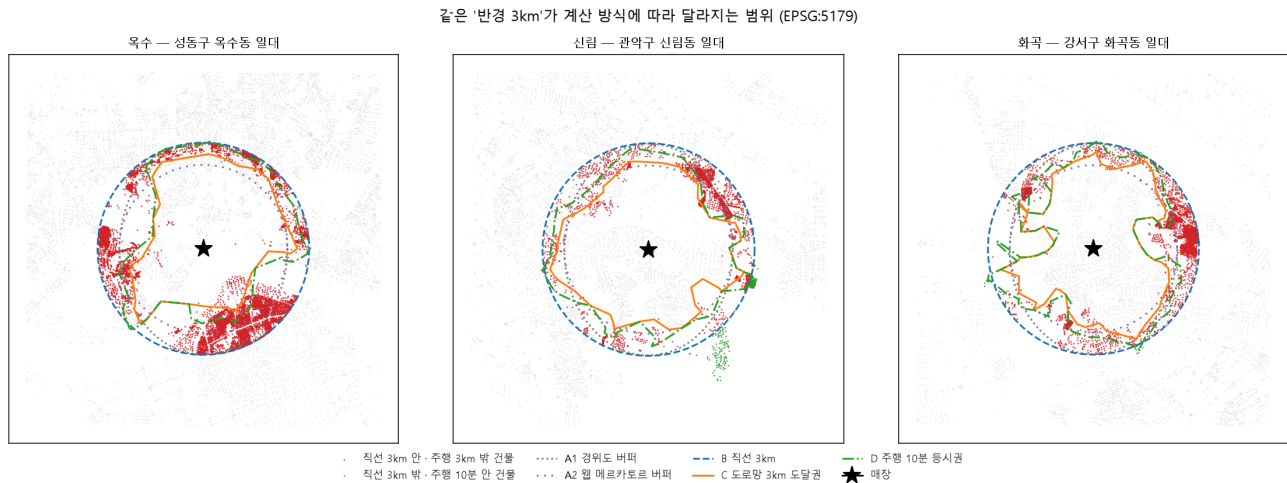


그림 2.3 같은 3km 조건도 좌표계와 거리 정의에 따라 권역 모양과 포함 건물 수가 달라짐. 빨간 점은 직선 3km 안이지만 주행 3km 밖인 건물, 초록 점은 직선 3km 밖이지만 주행 10분 안인 건물.

- 읽는 순서가 있음. A1·A2와 C·D는 B에서 벗어난 **이유가 서로 다름**
- **A1·A2는 좌표계를 잘못 쓴 결과** (4절의 함정)
- A1은 경위도에 도 단위 버퍼를 준 것 — 3km를 도로 환산한 0.02695°를 넣어도 동서 방향이 줄어들어 원이 타원이 됨
- A2는 웹 메르카토르에서 미터 버퍼를 준 것 — 건물 수가 기준의 **39%**. 위도가 올라갈수록 축척이 부풀려지는 성질을 무시하고 3,000을 넣었기 때문
- 둘 다 계산은 성공하고 지도에 그리면 원처럼 보임. 기준값과의 대조 없이는 잡히지 않음
- **C는 좌표계 문제가 아니라 거리 정의의 차이** (7절)
- 도로망으로 재면 건물이 2,418채로 기준의 **23%**
- 직선 3km 안이지만 주행 3km로는 못 가는 건물이 **7,925채(76.6%)** — 2절의 도입 사례가 이 숫자
- 이 건물들의 추가 주행거리는 중앙값 **720m**, 최대 **3,644m**. 권역 밖 수준의 주행을 "3km 안"으로 집계하고 있었던 것
- **D는 반대 방향의 예외를 보여 줌**
- 직선 3km 밖인데 10분 안에 도달하는 건물이 11채(기준의 0.1%). 큰길이 곧게 뚫린 방향으로만 생김

8.2 그래서 반경을 얼마로 정할까

- 직선 반경을 계속 쓰기로 했다면, 남은 결정은 "오배정(주행 3km 초과 건물이 권역에 들어오는 것)을 얼마나 감수할까"임
- 실측 결과의 킷라인:
- 오배정 0%까지 줄이려면 반경 187m
- 오배정 5% 이하면 반경 1,271m
- 187m는 매장 앞 몇 블록만 남는, 사실상 영업이 불가능한 반경
- 즉 직선 반경으로는 오배정을 완전히 없앨 수 없음. 도로가 조금이라도 우회하면 아주 가까운 건물에도 초과 주행이 생기기 때문
- 실무 선택지는 둘: 허용 오배정 비율(예: 5%)을 먼저 정하고 킷라인을 역산하거나, 아예 도로망 도달권으로 권역을 정의함
- 우회비를 알면 킷라인을 근사할 수도 있음: 근사 킷라인 = 3,000m ÷ 우회비 중앙값
- 신림은 근사 2,362m vs 실측 2,404m(차이 42m), 화곡은 근사 2,261m vs 실측 2,297m(차이 36m) — 우회가 사방으로 고른 곳에서는 우회비 하나로 충분함
- 옥수는 근사 2,009m vs 실측 1,271m로 738m 벌어짐 — 한강이 한 방향을 통째로 막아 우회가 한쪽에 몰렸기 때문
- 근사와 실측의 격차 자체가 진단 지표임. 격차가 크면 반경을 줄이는 대신 권역을 방향별로 나누는 편이 나음
- 하나 더 — 반경 3km 원 안에서 도로가 200m 안에 없는 구역이 14.0%(옥수)
- 산과 하천이 차지한 공간. 원으로 그린 권역에는 포함되지만 실제로는 배달할 집이 없음

9. 핵심 요약

- 좌표계는 숫자의 단위를 정함. 지리 좌표계(도)는 자료 교환용, 투영 좌표계(미터)는 계산용. 경위도 면적 9.9548도²는 어떤 실제 면적과도 비교할 수 없고, 등면적 투영으로 계산한 99,206km²가 쓸 수 있는 값 (4절, 그림 2.2)
- UTM은 존마다 평면 좌표를 매기고, 중심 경도에 500,000m를 부여함(False Easting). 부산의 동쪽 좌표 500,000.00은 우연이 아니라 존 중심에 있다는 뜻 (4.3절, 그림 2.1)
- 좌표 변환 자체는 정보를 잃지 않음. 역변환 오차는 사실상 0. 사고는 변환을 안 했거나 잘못된 좌표계로 했을 때 나며, 역변환 오차·축 순서·지도 육안의 세 가지로 검증함 (5절)
- 버퍼 거리의 단위는 좌표계가 정함. 경위도에서 buffer(3000)은 3,000도가 되고, 경고만 뜬 채 계산이 끝까지 뚫. 미터 좌표계로 옮긴 뒤 버퍼를 만드는 두 단계가 올바른 순서 (6.1절)

- 공간 조인 뒤에는 **행 수를 확인함**. 경계에 걸친 점은 행을 늘리고 빠진 점은 행을 줄이며, 둘 다 오류 없이 지나감 (6.3절)
- 도로망 거리는 직선보다 짧을 수 없으므로 직선 반경은 구조적으로 **과대 포함**. 과대의 크기를 재는 값이 우회비(주행÷직선)이며, 실측 중앙값은 옥수 1.493·신림 1.270·화곡 1.327로 지점마다 다름. 지형이 도로를 굽을수록 커짐 (7절)
- 같은 3km도 방식에 따라 다른 권역이 됨. 옥수에서 직선 3km 안 건물 10,343채 중 7,925채(76.6%)는 주행 3km 초과. 좌표계 오류(A1·A2)와 거리 정의 차이(C·D)는 벗어나는 이유가 서로 다름 (8.1절, 그림 2.3)
- 오배정은 0으로 만들 수 없고, 허용 비율을 정해 역산함. 옥수의 컷라인은 0%면 187m, 5%면 1,271m. 우회비로 낸 근사 컷라인과 실측의 격차가 크면 반경 축소 대신 방향별 권역 분할을 검토함 (8.2절)

10. 과제

- 이번 장의 과제는 직접 돌려 보고, 그 결과로 권역을 판단하는 것
- 명령 한 줄이면 제출할 파일이 생성됨

```
python scripts/submit.py 2 --id 학번 --name 이름
```

- 이 명령이 대신 처리하는 작업: 도로망 스냅샷 준비, 대표 실습 2-3-delivery-service-area.py 실행, 제출용 파일 생성
- 끝나면 submissions/ 폴더에 제출_2장_학번.md 파일이 생김
- 열어 보면 실행 결과가 이미 붙어 있음. 눈여겨볼 곳은 우회비(주행거리 ÷ 직선거리) 중앙값과 지점별 차이
- 맨 아래 빈칸 세 개를 채우면 됨
 1. 눈에 띈 숫자 하나를 그대로 옮겨 적고, 왜 눈에 띄었는지 한 문장으로
 2. 그 숫자가 왜 그렇게 나왔는가
 3. 매장 중심 반경 3km로 배달 권역을 그은 가게가 있다. 이 결과를 근거로 권역을 어떻게 바꾸라고 하겠는가?
- 3번이 이 과제의 핵심. 1·2번은 결과를 들여다보면 답이 나오지만, 3번은 8절의 판단 — 허용 오배정, 컷라인, 방향별 분할 — 을 자기 말로 내려야 함
- 채운 파일 하나만 올리면 제출이 끝남

- **막혀도 제출함**
- 실행이 도중에 실패하면 오류 메시지가 그 파일에 자동으로 담기고, 질문이 "어디서 막혔나"로 바뀜

- 어디서 왜 막혔는지 적은 제출도 온전한 제출. 실행 성공 여부로 점수를 매기지 않음

- **이 장의 과제는 학번을 시드로 실행됨**

- 스냅샷에서 바로 계산되는 값(권역 표, 우회비, 컷라인)은 교재와 같아야 정상
- 난수를 쓰는 부분(매장 위치 재배치 검사)의 값은 제출자마다 달라짐. 교재와 다르다고 오류가 아님